

From Yocto 5.2, there's no 'poky' distro directly anymore!!

[See this post from Richard Purdie on the OE mailing list for the details.](#)

Trying it out!

Step 1: Clone only BitBake itself

```
|yocto_newapppr_apr26 $ git clone https://git.openembedded.org/bitbake
```

Step 2: Run the interactive setup tool

```
|yocto_newapppr_apr26 $ bitbake/bin/bitbake-setup init
```

NOTE: Bitbake-setup is using /big/yocto_newapppr_apr26/bitbake-builds as top directory. /big/yocto_newapppr_apr26/bitbake-builds looks like a new top directory. If you would like to use a different directory, answer "n" below and either:

1) Change the default bitbake-setup settings:

bitbake-setup settings set default top-dir-prefix <PATH>

bitbake-setup settings set default top-dir-name <NAME>

2) Pass one or more options on the command line to change the top level directory in that invocation only:

bitbake-setup --setting default top-dir-prefix <PATH> ...

bitbake-setup --setting default top-dir-name <NAME> ...

A common site.conf file will be created, please check it is correct before running builds

/big/yocto_newapppr_apr26/bitbake-builds/site.conf

Proceed? (y/N): y

Available Configuration Templates:

1. oe-nodistro-master OpenEmbedded - 'nodistro' basic configuration
2. oe-nodistro-whinlatter OpenEmbedded - 'nodistro' basic configuration, release 5.3 'whinlatter' (supported until 2026-05-31)
3. poky-master Poky - The Yocto Project testing distribution configurations and hardware test platforms
4. poky-whinlatter Poky - The Yocto Project testing distribution configurations and hardware test platforms, release 5.3 'whinlatter' (supported until 2026-05-31)

Please select one of the above configurations by its number: 3

<<

For a custom image recipe to work, select opt 1??

To test...

>>

Available bitbake configurations:

1. poky Poky - The Yocto Project testing distribution
2. poky-with-sstate Poky - The Yocto Project testing distribution with internet sstate acceleration. Use with caution as it requires a completely robust local network with sufficient bandwidth.

Please select one of the above bitbake configurations by its number: 1

Target machines:

1. machine/qemux86-64 x86-64 system on QEMU
2. machine/qemuarm64 ARMv8 system on QEMU
3. machine/qemuriscv64 RISC-V system on QEMU
4. machine/genericarm64 Arm64 SystemReady IR/ES platforms
5. machine/genericx86-64 x86_64 (64-bit) PCs and servers

Please select one of the above options by its number: 2

Target distributions:

1. distro/poky Yocto Project Reference Distro
2. distro/poky-altcfg Poky alternative with systemd as init manager
3. **distro/poky-tiny** Poky alternative optimized for size

Please select one of the above options by its number: **3**

NOTE: Run 'bitbake-setup init --non-interactive poky-master poky distro/poky-tiny machine/qemuarm64' to select this configuration non-interactively.

Enter setup directory name [poky-tiny-master]:

Initializing a setup directory in
/big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master
Continue? (y/N): **y**

Fetching layer/tool repositories into /big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master/layers

bitbake
openembedded-core
meta-yocto
yocto-docs

Setting up bitbake configuration in
/big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master/build

This bitbake configuration provides:
Poky - The Yocto Project testing distribution

Usage instructions and additional information are in
/big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master/build/README

To run builds, source the environment using
. /big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master/build/init-build-env

Run 'bitbake-config-build enable-fragment <fragment-name>' to enable additional fragments or replace built-in ones (e.g. machine/<name> or distro/<name> to change MACHINE or DISTRO).

The bitbake configuration files (local.conf, bblayers.conf and more) can be found in
/big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master/build/conf

To edit the code in VSCode, open the workspace:
code /big/yocto_newappr_apr26/bitbake-builds/poky-tiny-master/bitbake.code-workspace
\$

```
|bitbake-builds $ tree -d -L 2 poky-tiny-master/
poky-tiny-master/
├── build
│   ├── cache
│   ├── conf
│   └── tmp
├── config
└── layers
    ├── bitbake
    ├── logs
    ├── meta-yocto
    ├── oe-init-build-env-dir -> openembedded-core
    ├── oe-scripts -> openembedded-core/scripts
    ├── openembedded-core
    └── yocto-docs
```

The VSCode workspace places you within the **build/conf** folder.

f.e. in bitbake-builds/poky-tiny-master/build

It also shows the layers:

meta-yocto, openembedded-core, yocto-docs, bitbake.

Step 3: Source the environment (note: path differs from old workflow!)

```
|bitbake-builds $ source poky-tiny-master/build/init-build-env
Poky - The Yocto Project testing distribution
|build $
```

Step 4: Build as before

```
|build $ time bitbake core-image-minimal
Loading cache: 100% |
#####
#####| Time: 0:00:00
Loaded 0 entries from dependency cache.
Parsing recipes: 100% |
#####
#####| Time: 0:00:19
Parsing of 950 .bb files complete (0 cached, 950 parsed). 1965 targets, 208 skipped, 0
masked, 0 errors.
NOTE: Resolving any missing task queue dependencies

Build Configuration:
BB_VERSION      = "2.16.0"
BUILD_SYS       = "x86_64-linux"
NATIVELSBSTRING = "ubuntu-24.04"
TARGET_SYS      = "aarch64-poky-linux-musl"
MACHINE         = "qemuarm64"
SDKMACHINE      = "x86_64"
DISTRO          = "poky-tiny"
DISTRO_VERSION  = "5.3.99+snapshot-8b5f530fc57ba0e04c47b2d7338f5c0f862b084f"
TUNE_FEATURES   = "aarch64 crc cortexa57"
meta            = "master:8b5f530fc57ba0e04c47b2d7338f5c0f862b084f"
meta-yocto-bsp  = "master:4fa3f79364c617f09185e39f1fd815287c687993"

NOTE: Fetching uninative binary shim
http://downloads.yoctoproject.org/releases/uninative/5.1/x86_64-nativesdk-libc-
5.1.tar.xz;sha256sum=2b63a078c26535e0786e87f81ae69509df30f4dce40693004c527bd5e4ab2b85
(will check PREMIRRORS first)
Sstate summary: Wanted 1764 Local 0 Mirrors 0 Missed 1764 Current 0 (0% match, 0%
complete)
Initialising tasks: 100% |
#####
#####| Time: 0:00:02
NOTE: Executing Tasks
WARNING: linux-yocto-tiny-6.18.19+git-r0 do_fetch: Failed to fetch URL
git://git.yoctoproject.org/linux-yocto.git;branch=v6.18/standard/tiny/
base;name=machine;protocol=https, attempting MIRRORS if available
NOTE: Tasks Summary: Attempted 3312 tasks of which 0 didn't need to be rerun and all
succeeded.

Summary: There was 1 WARNING message.

real    144m26.080s
user    0m16.898s
sys     0m2.976s
|build $
```

Config fragments!

When you set up the build environment, a configuration file named *toolcfg.conf* becomes available in a *conf/* subdirectory of the BitBake build directory, automatically managed by the bitbake-config-

build command-line tool. This sets up the environment similarly to what was previously in local.conf, which is now largely empty.

You can enable build options via fragments instead of manually editing local.conf:

```
# Example: allow root login for development
$ bitbake-config-build enable-fragment root-login-with-empty-password
```

This is a significant philosophical shift — build configuration is now expressed as composable, named fragments rather than ad-hoc edits to a flat config file.

File that gets updated:

bitbake-builds/poky-tiny-master/build/conf/toolcfg.conf

```
$ cat bitbake-builds/poky-tiny-master/build/conf/toolcfg.conf
Automated config file controlled by tools
#
# Run 'bitbake-config-build enable-fragment <fragment-name>' to enable additional
# fragments
# or replace built-in ones (e.g. machine/<name> or distro/<name> to change MACHINE or
# DISTRO).
```

```
OE_FRAGMENTS += "distro/poky-tiny machine/qemuarm64 core/yocto/root-login-with-empty-
password"
```

This got added...

FAQ>

From Yocto 5.3, we can use **config fragments** to enable stuff, instead of bloating the *local.conf* file. Now, show me:

1. Which config fragments / features can be enabled? earlier i would use
EXTRA_IMAGE_FEATURES += "dbg-pkgs tools-debug debug-tweaks" etc
2. Provide a full list of all features that can be enabled/disabled in this manner pl.

ANS> Courtesy claud.ai: [Link](#).

V IMP:

Yocto Features – Machine, Distro, Image (& Backfilling).

[A summary of the same \(courtesy ChatGPT\).](#)

FAQ> **Build Env sourcing:** *when I want to do the familiar*
cd poky; . oe-init-build-env <build-dir>
to start working, it isn't there??

A> *It is there but in a slightly different manner, via a small wrapper script under your build-dir named init-build-env.*

```
cd bitbake-builds/poky-tiny-master/build
```

```
|build $ ls
```

```
bitbake-cookerdaemon.log  cache/  conf/  init-build-env  README  tmp/
```

```
|build $ . init-build-env  
Poky - The Yocto Project testing distribution  
|build $
```

All set! F.e.:

```
|build $ echo $BUILDDIR  
/<...>/bitbake-builds/poky-tiny-master/build  
|build $ bitbake-getvar TOPDIR  
NOTE: Starting bitbake server...  
#  
# $TOPDIR [2 operations]  
#   set cookerdata.py:370 [parseConfigurationFiles]  
#     "<...>/bitbake-builds/poky-tiny-master/build"  
#   set <...>//bitbake-builds/poky-tiny-master/layers/openembedded-core/meta/conf/  
documentation.conf:430  
#     [doc] "The Build Directory. BitBake automatically sets this variable. The  
OpenEmbedded build system uses the Build Directory when building images."  
# pre-expansion value:  
#   "<...>/bitbake-builds/poky-tiny-master/build"  
TOPDIR="<...>/bitbake-builds/poky-tiny-master/build"
```

```
...  
./bitbake/bin/bitbake-setup init NOTE: Bitbake-setup is using  
/big/yocto_newappr_apr26/bitbake-builds as top directory. NOTE: A site.conf file  
already exists. Please remove it if you would like to replace it with a default  
one
```

Ah, this is new!

Bitbake sets up a system (host)-wide *site.conf* file the first time you perform an 'init'. So you should continue to work there (in the original workspace), just creating a new folder for a new build...

Practical :

Buiding for **TI Beagleplay**: - several layer dependencies on the MACHINE !
([ref article](#))

```
cd <...>/bitbake-builds/poky-altcfg-bglplay/layers
```

```
git clone git://git.yoctoproject.org/meta-arm  
git clone git://git.yoctoproject.org/meta-ti
```

```
cd -  
bitbake-layers add-layer ../layers/meta-arm/meta-arm  
bitbake-layers add-layer ../layers/meta-arm/meta-arm-toolchain  
bitbake-layers add-layer ../layers/meta-ti/meta-ti-bsp/  
bitbake-layers add-layer ../layers/meta-ti/meta-beagle/
```

```
$ bitbake-layers show-layers  
NOTE: Starting bitbake server...  
layer          path  
priority
```

```

=====
=====
core                <...>/bitbake-builds/poky-altcfg-bglplay/layers/openembedded-
core/meta  5
yoctobsp          <...>/bitbake-builds/poky-altcfg-bglplay/layers/meta-yocto/meta-
yocto-bsp  5
yocto            <...>/bitbake-builds/poky-altcfg-bglplay/layers/meta-yocto/meta-
poky  5
meta-mydistro     <...>/bitbake-builds/poky-altcfg-bglplay/build/meta-mydistro  6
arm-toolchain    <...>/bitbake-builds/poky-altcfg-bglplay/layers/meta-arm/meta-arm-
toolchain  5
meta-arm         <...>/bitbake-builds/poky-altcfg-bglplay/layers/meta-arm/meta-arm
5
meta-ti-bsp       <...>/bitbake-builds/poky-altcfg-bglplay/layers/meta-ti/meta-ti-
bsp  6
meta-beagle       <...>/bitbake-builds/poky-altcfg-bglplay/layers/meta-ti/meta-
beagle  6

```

```

|build $ bitbake -e | grep -E "^MACHINE=|DISTRO_VERSION=|DISTRO="
DISTRO="mydistro"
DISTRO_VERSION="1.0"
INHERIT_DISTRO="debian devshell sstate license remove-libtool create-spx buildstats
uninative"
MACHINE="beagleplay"

```

...

bitbake core-image-minimal

...

Took a while (>3 hours)..

Images found not under tmp/deploy but under deploy-ti/images/beagleplay/ !

Yocto for TI BeagleBone Black (BBB) – new approach

My workspace: yocto_bbb

mkdir layers

Run this script *just once* to initialize the workspace.

```
$ cat new_init_no_poky.sh
```

```
#!/bin/bash
```

```
# Correct way to setup the Yocto >=5.3 layers directory layout
```

```
UPSTREAM_LAYERS_DIR=upstream_layers
```

```
[[ ! -d ${UPSTREAM_LAYERS_DIR} ]] && mkdir ${UPSTREAM_LAYERS_DIR}
```

```
git clone -b yocto-5.3.3 https://git.openembedded.org/bitbake
./${UPSTREAM_LAYERS_DIR}/bitbake
git clone -b yocto-5.3.3 https://git.openembedded.org/openembedded-core ./${
UPSTREAM_LAYERS_DIR}/openembedded-core
git clone -b yocto-5.3.3 https://git.yoctoproject.org/meta-yocto ./${
UPSTREAM_LAYERS_DIR}/meta-yocto
git clone -b yocto-5.3.3 https://git.yoctoproject.org/yocto-docs ./${
UPSTREAM_LAYERS_DIR}/yocto-docs
```

```
# Create and maintain your own layers in another dir (which is version
# controlled, f.e. via git)
# f.e.
# MYPRJ_LAYERS=myprj_layers
```

Source the environment – every time!

```
$ source layers/openembedded-core/oe-init-build-env build_bbb
...
```

You can also run generated qemu images with a command like 'runqemu qemux86-64'.

Other commonly useful commands are:

- 'devtool' and 'recipetool' handle common recipe tasks
- 'bitbake-layers' handles common layer tasks
- 'oe-pkgdata-util' handles common target package tasks

You had no /big/yocto_bbb_new/layers/openembedded-core/.vscode configuration.
These configuration files have therefore been created for you.
|build_bbb \$

```
$ tree .
```

```
├── conf
│   ├── bblayers.conf
│   ├── conf-notes.txt
│   ├── conf-summary.txt
│   ├── local.conf
│   └── templateconf.cfg
```

Add the required layers!

```
bitbake-layers add-layer ../upstream_layers/meta-yocto/meta-yocto-bsp/
bitbake-layers add-layer ../upstream_layers/meta-yocto/meta-poky/
```

```
$ bitbake-layers show-layers
```

```
NOTE: Starting bitbake server...
```

```
layer                path
```

```
priority
```

```
=====
=====
core                /big/yocto_bbb_new/upstream_layers/openembedded-core/meta      5
yoctobsp            /big/yocto_bbb_new/upstream_layers/meta-yocto/meta-yocto-bsp    5
yocto               /big/yocto_bbb_new/upstream_layers/meta-yocto/meta-poky        5
...
```

V similar to pre 5.3 now!

```
|build_bbb $ bitbake -e core-image-minimal |grep "^DISTRO.*="
```

```
ERROR: OE-core's config sanity checker detected a potential misconfiguration.
```

```
Either fix the cause of this error or at your own risk disable the checker (see
sanity.conf).
```

```
Following is the list of potential problems / advisories:
```

```
MACHINE=beaglebone-yocto is invalid. Please set a valid MACHINE in your local.conf,
environment or other configuration file.
```

Missing layers! Add them:

```
cd ../upstream_layers
git clone git://git.yoctoproject.org/meta-arm
git clone git://git.yoctoproject.org/meta-ti

cd ../upstream_layers/meta-arm
git checkout whinlatter # << can replace this w/ the branch you're using >>

cd -
bitbake-layers add-layer ../upstream_layers/meta-arm/meta-arm
bitbake-layers add-layer ../upstream_layers/meta-arm/meta-arm-toolchain
bitbake-layers add-layer ../upstream_layers/meta-ti/meta-ti-bsp/
bitbake-layers add-layer ../upstream_layers/meta-ti/meta-beagle/
```

```
|build_bbb $ bitbake-layers show-layers |awk 'NR > 3 {printf "%20s %50s %5d\n", $1, substr($2,20), $3}'
```

core	upstream_layers/openembedded-core/meta	5
yoctobsp	upstream_layers/meta-yocto/meta-yocto-bsp	5
yocto	upstream_layers/meta-yocto/meta-poky	5
arm-toolchain	upstream_layers/meta-arm/meta-arm-toolchain	5
meta-arm	upstream_layers/meta-arm/meta-arm	5
meta-ti-bsp	upstream_layers/meta-ti/meta-ti-bsp	6
meta-beagle	upstream_layers/meta-ti/meta-beagle	6
meta-mydistro	myprj_layers/meta-mydistro	6
meta-myprj	myprj_layers/meta-myprj	6
meta-helloworld	myprj_layers/meta-helloworld	6
meta-kmods	myprj_layers/meta-kmods	10
meta-foo	myprj_layers/meta-foo	6

Nice! See how the Yocto upstream layers are separated out from the project layers.
Build it!

```
$ bitbake core-image-minimal
```

```
...
...
```

Succeeds.

TIP: Keep your project layers in it's own folder.

So, for example:

```
|yocto_bbb_new $ tree -d -L 1 upstream_layers/
```

```
upstream_layers
├── bitbake
├── meta-arm
├── meta-ti
├── meta-yocto
├── openembedded-core
└── yocto-docs
```

```
7 directories
```

```
|yocto_bbb_new $ tree -d -L 1 myprj_layers/ << version-controlled, your project repo >>
```

```
myprj_layers/
├── meta-foo
├── meta-helloworld
├── meta-kmods
└── meta-mydistro
```

Best to keep all the upstream meta layers in a separate folder and your layers under the root of your source repo; as one example, the [OpenBMC project](#) does just this:


```
|openbmc $ ls
bitbake@      meta-amd/      meta-bytedance/  meta-google/    meta-intel-
openbmc/      meta-openpower/  meta-security@   meta-yadro/     README.md
build/        meta-ampere/     meta-delta/      meta-hpe/       meta-inventec/
meta-phosphor/ meta-supermicro/ oe-init-build-env@ scripts@
eslint.config.js meta-arm@        meta-evb/        meta-ibm/       meta-nuvoton/
meta-qualcomm/ meta-tyan/       openbmc-env      setup*
LICENSE@      meta-aspeed/    meta-facebook/   meta-ieisystem/ meta-nvidia/
meta-quanta/  meta-ufispace/  OWNERS           upstream-layers/
meta@         meta-asrock/    meta-fii/        meta-ingrasys/
meta-openembedded@ meta-raspberrypi@ meta-wistron/    poky/
|openbmc $
|openbmc $ ls upstream-layers/
bitbake/ meta-arm/ meta-openembedded/ meta-raspberrypi/ meta-security/
openembedded-core/ README yocto-docs/
```

Recommended Way to setup the Yocto workspace – using a distro + image recipe in production projects

Partial courtesy: claude.ai

...

So is `IMAGE_INSTALL:append` in `local.conf` Still Valid?

Yes, fully valid and unchanged in 5.3. Nothing about the bitbake-setup / fragments overhaul touched image content control. You can still do:

```
# conf/local.conf
IMAGE_INSTALL:append = " ltrace strace"
```

And it works exactly as before. For quick dev iteration or a personal workspace, this is perfectly fine.

But What's the Proper Recommended Way?

The Yocto community's long-standing recommendation (not new to 5.3) is that `local.conf` edits are **ephemeral developer conveniences**, not shareable, not version-controlled, not reproducible across team members.

<<

The real Yocto philosophy: **machine × distro × image are three orthogonal axes**

>>

The cleaner patterns, in order of preference for a real project:

Pattern 1 — Custom image recipe (best for production)

Create your own image recipe in your BSP/product layer:

```
# meta-myproduct/recipes-core/images/my-product-image.bb
require recipes-core/images/core-image-minimal.bb

IMAGE_INSTALL:append = " ltrace strace gdbserver"
SUMMARY = "My product image"
```

Then build `my-product-image` instead of `core-image-minimal`. This is version-controlled, reproducible, and the image definition travels with your layer.

Pattern 2 — `.bbappend` on an existing image (surgical override)

If you don't want to own a whole new image recipe:

```
# meta-myproduct/recipes-core/images/core-image-minimal.bbappend
IMAGE_INSTALL:append = " ltrace strace"
```

This non-destructively extends the upstream image from within your layer — same philosophy as patching a recipe without forking it.

Pattern 3 — Wrapping in a config fragment (valid but narrow use case)

You *can* technically write a fragment file that sets `IMAGE_INSTALL`:

```
# meta-myproduct/conf/fragments/dev-debug-tools.conf
IMAGE_INSTALL:append = " ltrace strace"
```

And enable it via:

```
bitbake-config-build enable-fragment dev-debug-tools
```

But this is an awkward fit. Fragments are designed for coarse build-env knobs, not package lists — and **importantly, a fragment applies globally to every image you build in that setup**, not to a specific image. For a `ltrace/strace` use case, this is the right answer only if you want a **workspace-level dev overlay** (e.g., always add debug tools in this build dir regardless of which image you're building). It's not the canonical place for image content.

Decision Guide

Are you hacking locally, just for yourself today?

→ `local.conf`, `IMAGE_INSTALL:append`. Quick, fine.

Is this a permanent part of your product?

→ Custom image recipe (`.bb`) in your product layer.

Are you extending an upstream image minimally?

→ `.bbappend` in your layer.

Do you want a dev-mode workspace overlay (debug tools in every image in this `buildddir`)?

→ Config fragment in your layer, enabled via `bitbake-config-build`.

The `local.conf` approach isn't wrong — **it's just the lowest tier on the reproducibility and shareability ladder**. The 5.3 changes didn't demote it; they just gave the build-environment configuration tier a proper home (`fragments/toolcfg.conf`), which was previously also slopped into `local.conf` alongside image content. Those two concerns are now cleanly separable, but `IMAGE_INSTALL` itself hasn't moved.

That last point is the real Yocto philosophy: **machine × distro × image are three orthogonal axes**, and `local.conf` is just the place where you pin a specific point in that 3-space for *this build dir, right now*. Refactoring out of `local.conf` is the act of moving each variable to the axis it actually belongs on.
